

# Sina Hazeghi

[shazeghian@gmail.com](mailto:shazeghian@gmail.com) | [github.com/Sina-Haz](https://github.com/Sina-Haz) | [sina-haz.github.io](https://sina-haz.github.io)

## EDUCATION

**University of Pennsylvania | Philadelphia, PA**

**Expected December 2027**

- Master of Science in Engineering, Robotics
- Coursework: Machine Perception, Control and Optimization, Graph Neural Networks, Intro to Mechatronics

**Rutgers University | New Brunswick, NJ**

**December 2024**

- Bachelor of Science in Computer Science and Mathematics, *Summa Cum Laude*
- GPA: 3.96/4.00 ; SAS Academic Excellence Scholarship Recipient
- Coursework: AI, Computational/Advanced Robotics, Stochastic Processes, Theory of Statistics, Numerical Analysis

## SKILLS

- Languages: Java, Python, C/C++, JavaScript, Typescript, HTML, CSS, X86 Assembly, SQL, R
- Technologies: AWS, CUDA, Git, JAX, Docker, GCP, Pandas, PyTorch, HuggingFace, fast.ai, Transformers, Access DB

## WORK EXPERIENCE

**BNY** **August 2025 - August 2026**

Software Engineer

New York, NY

- Developed an agentic AI orchestration engine ("Eliza Holmes") using CrewAI and MongoDB to dynamically instantiate agentic workflows to accelerate client onboarding by up to 90% and winning BNY's 2025 technical innovation award
- Designed a token-aware hierarchical document indexing engine with LLM summary trees and MongoDB tree-caching, cutting LLM context consumption by 70–90% and query latency from 30s to <20ms during bulk entity onboarding
- Implemented a pluggable multi-tiered guardrail framework incorporating dynamic Python validators, thread-safe execution tracking, and adversarial judge agents, catching 83% of output errors prior to end-user review
- Engineered enterprise Java microservices using Spring Boot, GraphQL, and OracleDB to empower agents to integrate seamlessly with legacy databases and existing KYC/AML onboarding infrastructure
- Built an Angular prompt-management platform for developers to analyze runtime context, rapidly fine-tune agentic prompts, and apply hot-fixes without CI/CD redeployments—resolving ~70% of user-reported KYC workflow bottlenecks
- Diagnosed bulk onboarding failures during 10x document scaling as agentic iteration budget exhaustion; resolved by engineering multi-threaded batch processing to duplicate crew instances across parallel workloads, stripping irrelevant context to eliminate execution errors and accelerate bulk onboarding workflows by 35%

**Rutgers University - [PRACSYS Lab](#)**

**December 2023 – May 2025**

Student Researcher

New Brunswick, NJ

- Evaluated differentiable physics backends (Taichi, NVIDIA Warp, JAX/Jraph) for simulating NASA's cable-driven Tensegrity robot, converging on a fully learned JAX/Jraph simulator that runs 100x faster than MuJoCo
- Optimized Graph Neural Network (GNN) inference by 1.5x using JAX/Jraph over baseline PyTorch, enabling downstream research in MPC and closed-loop control for terrain-based robotics applications
- Trained simulators under partially-observable state data for real2sim transfer, informing research direction later formalized in "CableRobotGraphSim," accepted for oral presentation at L4DC 2026

**Pfizer**

**May 2024 - August 2024**

Process Development Intern

Pearl River, NY

- Deployed Shiny for Python website to automate inventory processes and database integration saving 2-3 hours/shipment
- Updated R-Shiny app and underlying SQL database by merging subroutines to add image compatibility, PDF integration with LaTeX, and dynamically populated protocols, simplifying analyst workflows by 60%

## PROJECTS

**Maze Solver Motion Generation Implementations**

- Devised a custom task in Python and MuJoCo for a robot to navigate a maze of obstacles from random start points
- Developed Kinodynamic RRT planner with a 96% success rate, achieving navigation on average of 5.08 seconds
- Devised GPU-accelerated training scripts using Torch RL to leverage distributed training to reduce training time by 95%
- Implemented A2C, PPO and SAC algorithms with various advantage estimation and policy formulation methods from scratch in PyTorch to maximize successful navigation
- Tuned algorithms to surpass baseline implementations, achieving a 22% higher success rate after 10,000 training episodes, with successes taking on average only 2.38 seconds